

1 Problem 4: Critical-Word First and Early Restart (16 points)

Common setup and notation. A word is 8 Bytes, so a block of B Bytes holds $n = B/8$ words. For a given cache level write

t_1 time until the *first* word arrives when the block is sent in normal order (no critical-word first);

t_x time for each *additional* word;

t_c time until the requested word arrives when critical-word first is used (it is sent first, so this single transfer is all the processor waits for).

Without reordering, word i of the block ($i = 0, 1, \dots, n-1$) is available after $t_1 + i t_x$. The three schemes are then

$$\text{neither: } T_{\text{none}} = t_1 + (n-1)t_x \quad (\text{wait for the whole block}) \quad (1)$$

$$\text{early restart only: } T_{\text{ER}} = \sum_{i=0}^{n-1} p_i (t_1 + i t_x) \quad (\text{resume when word } i \text{ arrives}) \quad (2)$$

$$\text{both: } T_{\text{both}} = t_c \quad (\text{requested word first, then resume}) \quad (3)$$

where p_i is the probability that the referenced word sits at position i . Equations (1) and (3) do not involve p_i : in the first the processor waits for the last word regardless of what it asked for, and in the third the requested word is always the first one delivered. Only (2) is a genuine expectation over the reference distribution.

Question A (8 points)

Here $B = 64$ Bytes, so

$$n = \frac{64 \text{ Bytes}}{8 \text{ Bytes/word}} = 8 \text{ words per block,}$$

with $t_1 = 90$, $t_x = 10$ and $t_c = 110$ CCs. (The 1 MB capacity is not needed: the miss service time depends on the block size, not on how many blocks the cache holds.) Word i therefore arrives at $90 + 10i$ CCs, i.e. at 90, 100, 110, 120, 130, 140, 150, 160 CCs for $i = 0, \dots, 7$.

The reference distribution is *not* uniform. The first word carries 30% and the remaining $n-1 = 7$ words share the other 70% equally:

$$p_0 = 0.30, \quad p_i = \frac{0.70}{7} = 0.10 \quad (i = 1, \dots, 7), \quad \text{check: } 0.30 + 7(0.10) = 1.$$

(1) Neither critical-word first nor early restart. The processor resumes only once the entire block has arrived, which happens when the last word lands:

$$T_{\text{none}} = t_1 + (n-1)t_x = 90 + 7 \times 10 = \boxed{160 \text{ CCs}}.$$

This does not depend on which word was requested, so the reference distribution is irrelevant here and the “average” equals this fixed value.

(2) Early restart only. The block still arrives in normal order, but the processor resumes as soon as the requested word has arrived, so the wait depends on that word’s position:

$$\begin{aligned} T_{\text{ER}} &= 0.30 \times 90 + 0.10(100 + 110 + 120 + 130 + 140 + 150 + 160) \\ &= 27 + 0.10 \times 910 = 27 + 91 = \boxed{118 \text{ CCs}}. \end{aligned}$$

(3) Both critical-word first and early restart. Memory delivers the requested word first, at a cost of $t_c = 110$ CCs, and the processor restarts immediately:

$$T_{\text{both}} = 110 \text{ CCs} \Rightarrow \boxed{110 \text{ CCs}}.$$

Again the block-position distribution drops out entirely: whichever word is requested, it is the one sent first, so every miss costs the same 110 CCs. Note that critical-word first is *more expensive per transfer* than a normal first word ($110 > 90$); it still wins overall because it removes the wait for all seven remaining words.

Summary.

Scheme	Average L2 miss service time	Saving vs. (1)
(1) Neither	160 CCs	—
(2) Early restart only	118 CCs	$42/160 = 26.25\%$
(3) Critical-word first + early restart	110 CCs	$50/160 = 31.25\%$

Question B (4 points)

Answers: the first blank is 18 CCs and the second blank is 10%.

Now $B = 32$ Bytes, so

$$n = \frac{32 \text{ Bytes}}{8 \text{ Bytes/word}} = 4 \text{ words per block},$$

with $t_x = 4$ CCs, $t_c = 27$ CCs, and every word equally likely, $p_i = \frac{1}{4}$. Let x denote the unknown first-word latency.

First blank — recovered from the stated 20%. The paragraph gives us the early-restart result and asks us to work backwards. In terms of x ,

$$\begin{aligned} T_{\text{none}} &= x + (n - 1)t_x = x + 3 \times 4 = x + 12, \\ T_{\text{ER}} &= \frac{1}{4}[x + (x + 4) + (x + 8) + (x + 12)] = x + \frac{4(0 + 1 + 2 + 3)}{4} = x + 6. \end{aligned}$$

Early restart alone reduces the original time by 20%, so

$$\frac{T_{\text{none}} - T_{\text{ER}}}{T_{\text{none}}} = \frac{(x + 12) - (x + 6)}{x + 12} = \frac{6}{x + 12} = 0.20 \Rightarrow x + 12 = 30 \Rightarrow \boxed{x = 18 \text{ CCs}}.$$

Hence $T_{\text{none}} = 30$ CCs and $T_{\text{ER}} = 24$ CCs, which indeed differ by $6/30 = 20\%$.

Second blank. With critical-word first the requested word arrives after $t_c = 27$ CCs and the processor restarts there, so $T_{\text{both}} = 27$ CCs and

$$\frac{T_{\text{none}} - T_{\text{both}}}{T_{\text{none}}} = \frac{30 - 27}{30} = \frac{1}{10} = \boxed{10\%}.$$

Both blanks are exact integers, as the question requires.

Question C (4 points)

Verdict. On these numbers the two techniques are **more important for the L2 cache**.

Reason (from the results of A and B). Comparing the savings the two techniques actually deliver at each level:

	L2 (Question A)	L1 (Question B)
Words per block	8	4
Neither	160 CCs	30 CCs
Early restart only	118 CCs (−26.25%)	24 CCs (−20%)
Critical-word first + early restart	110 CCs (−31.25%)	27 CCs (−10%)
Absolute saving of the pair	50 CCs	3 CCs

At L2 the pair of techniques removes 31.25% of the miss service time (50 CCs); at L1 it removes only 10% (3 CCs). The gap is even sharper for critical-word first taken on its own: at L2 it improves on early restart alone (118 → 110 CCs), whereas at L1 it makes matters *worse* (24 → 27 CCs), because its $27 - 18 = 9$ CC surcharge exceeds the 6 CC that early restart had already saved. So at L1 it would be better not to enable critical-word first at all, while at L2 it pays for itself — which is precisely what “more important for L2” means.

Factors that drive the relative importance.

1. *Words per block, n .* The most these techniques can ever save is the transfer tail $(n - 1)t_x$ — the time spent shipping the words the processor is not waiting for. Larger blocks mean a longer tail and more to skip. Lower cache levels normally use larger blocks (8 words here versus 4), which favours L2.
2. *Fixed first-word latency versus the tail.* The ceiling on the saving is $(n - 1)t_x / [t_1 + (n - 1)t_x]$: $70/160 = 43.75\%$ at L2 and $12/30 = 40\%$ at L1. When the fixed latency dominates, the processor must wait for it anyway and neither technique can help much. Here the two levels start out comparable, so this is not what separates them.
3. *The surcharge critical-word first itself costs.* Reordering the transfer is not free: $t_c - t_1$ is 20 CCs at L2 but 9 CCs at L1. Relative to the block service time that is $20/160 = 12.5\%$ at L2 against $9/30 = 30\%$ at L1. This is the decisive factor: it consumes 12.5 of L2’s 43.75 available points but 30 of L1’s 40, leaving 31.25% and 10% respectively. The shorter a level’s total miss time, the harder a fixed reordering overhead is to justify.
4. *Shape of the reference distribution.* Early restart only pays off to the extent that the requested word arrives early. L2’s 30% weight on the first word helps it: under a uniform $1/8$ the L2 early-restart average would be 125 CCs, a saving of 21.875% rather than 26.25%. Critical-word first is immune to this, since its cost is flat in the word position — which makes it the more attractive of the two when the distribution is unfavourable or unknown.
5. *Miss frequency (a full-AMAT caveat).* The comparison above is *per miss*, which is what this problem measures. A complete average-memory-access-time analysis would also weight each level’s saving by how often that level misses, and L1 misses are far more frequent than L2 misses. That is the one factor that could push the balance back towards L1; with the numbers given here, though, L1’s saving is only 3 CCs against L2’s 50 CCs, so it would take an L1:L2 miss ratio above roughly 16:1 merely to draw level.